UNIVERSIDADE CATÓLICA DE BRASÍLIA
Engenharia de Software — Estrutura de Dados

DOCUMENTAÇÃO TÉCNICA

Sistema de Dados — Tribunais Eleitorais

Consolidação CNJ — Metas 2026

ANITA SILVA MOREIRA | UC2510177
DIOGO SOUZA MOREIRA | UC25102288
ERICK TEIXEIRA DOS SANTOS | UC25101750
GUILHERME AUGUSTO DOS SANTOS | UC25100900
JOÃO VICTOR VERAS AGAPITO | UC25102027

Prof. Marcelo Eustaquio
Brasília, 2026

1. Visão Geral do Projeto

O **Sistema de Dados — Tribunais Eleitorais** é um programa em linguagem C desenvolvido como trabalho bimestral da disciplina de Estrutura de Dados. O sistema lê, concatena e manipula arquivos CSV contendo dados dos Tribunais Regionais Eleitorais (TREs) de todos os estados brasileiros, fornecendo uma interface de menu para três operações principais.

Ambiente de desenvolvimento: Linux Zorin OS. Por esse motivo, a função `LimparTerminal()` detecta o sistema operacional em tempo de compilação via diretiva de pré-processador `#ifdef _WIN32` e executa `cls` no Windows ou `clear` no Linux/macOS. Caso o programa seja compilado em Windows, ele funcionará corretamente, mas foi otimizado e testado no ambiente Linux.

1.1 Arquitetura Geral

O projeto é composto por três arquivos:

- main.c — ponto de entrada, loop de menu e chamadas às funções principais
- lista.h — definição das estruturas de dados (Tribunal e Lista) e declaração de todas as funções
- lista.c — implementação de todas as funções auxiliares e principais

1.2 Estruturas de Dados

struct Tribunal

Representa um único registro de um órgão julgador eleitoral. Contém 33 campos que espelham as colunas do CSV:

Campo	Tipo	Descrição
`sigla_tribunal`	`char[8]`	Sigla do tribunal (ex: TRE-DF)
`procedimento`	`char[30]`	Tipo de procedimento processual
`ramo_justica`	`char[20]`	Ramo da justiça
`sigla_grau`	`char[4]`	Grau de jurisdição (ex: G1)
`uf_oj`	`char[4]`	UF do órgão julgador
`municipio_oj`	`char[35]`	Município do órgão julgador
`id_ultimo_oj`	`int`	Identificador numérico do órgão julgador
`nome`	`char[70]`	Nome completo da zona eleitoral
`mesano_cnm1`	`char[10]`	Mês/ano de referência CNM1 (pode ser NA)
`mesano_sent`	`char[10]`	Mês/ano da sentença (pode ser NA)
`casos_novos_2026`	`int`	Número de casos novos em 2026
`julgados_2026`	`int`	Número de casos julgados em 2026
`cumprimento_meta1`	`float`	Percentual de cumprimento da Meta 1
`distm2_a / julgm2_a`	`int`	Distribuídos e julgados da Meta 2A
`distm2_ant / julgm2_ant`	`int`	Distribuídos e julgados da Meta 2 Anterior

`distm4_a / julgm4_a`	`int`	Distribuídos e julgados da Meta 4A
`distm4_b / julgm4_b`	`int`	Distribuídos e julgados da Meta 4B
`cumprimento_meta4b`	`float`	Percentual de cumprimento da Meta 4B

struct Lista

Contém um array estático de `MAX_TRIBUNAIS` (definido como 40.000 em `lista.h`) elementos do tipo `Tribunal`, além de `Tamanho` (número de elementos atualmente carregados) e `Capacidade` (limite máximo).

Atenção: Por ser declarada como variável global em `main.c` (`Lista L;`), a lista é alocada na memória BSS/data do processo e não na stack, evitando stack overflow — que ocorreria caso fosse declarada localmente dada a sua grande dimensão.

2. Configuração e Requisitos

2.1 Compilação

Compile todos os arquivos juntos com gcc:

```
gcc -o tribunal main.c lista.c -lm
```

A flag -lm vincula a biblioteca matemática, necessária para operações com float.

2.2 Localização dos Arquivos CSV

Todos os arquivos CSV de entrada devem estar no mesmo diretório do executável. O programa busca os arquivos pelo nome exato, sem caminho relativo ou absoluto.

Os 27 arquivos de entrada esperados são nomeados no padrão:

```
teste_TRE-AC.csv
teste_TRE-AL.csv
teste_TRE-AM.csv
```

... e assim por diante para todos os estados: AP, BA, CE, DF, ES, GO, MA, MG, MS, MT, PA, PB, PE, PI, PR, RJ, RN, RO, RR, RS, SC, SE, SP, TO.

Se algum arquivo não for encontrado, o programa imprime uma mensagem de erro via `perror()` e pula aquele arquivo, continuando com os demais.

2.3 Arquivos Gerados

Opção 1 → Tribunais_Concatenados.csv	Todos os 27 CSVs unidos em um único arquivo com cabeçalho
Opção 2 → Resumo_Tribunais.csv	Uma linha por tribunal com totais e percentuais das metas

Opção 3 → <MUNICIPIO>.csv	Arquivo com todos os registros do município pesquisado (ex: BRASILIA.csv)

3. Como Usar o Programa

3.1 Menu Principal

Ao executar o programa, o seguinte menu é exibido:

```
=======================================================
       SISTEMA DE DADOS - TRIBUNAIS ELEITORAIS
             Consolidacao CNJ - Metas 2026
=======================================================

  [1]  Concatenar todos os CSVs em um unico arquivo
  [2]  Gerar resumo de metas por tribunal
  [3]  Pesquisar registros por municipio
  [0]  Sair
```

3.2 Opção 1 — Concatenar CSVs

Lê os 27 arquivos de entrada em sequência e grava todas as linhas em um único arquivo Tribunais_Concatenados.csv, precedido de um único cabeçalho. O progresso é exibido linha a linha no terminal.

3.3 Opção 2 — Gerar Resumo

Lê cada tribunal, soma os valores de todos os seus órgãos julgadores e calcula os percentuais das cinco metas (Meta 1, Meta 2A, Meta 2 Anterior, Meta 4A e Meta 4B) usando as fórmulas:

```
Meta 1   = julgados / (casos_novos + dessobrestados - suspensos) × 100
Meta 2A  = julgm2_a / (distm2_a - suspm2_a) × (1000 / 7)
Meta 2Ant= julgm2_ant / (distm2_ant - suspm2_ant - desom2_ant) × 100
Meta 4A  = julgm4_a / (distm4_a - suspm4_a) × 100
Meta 4B  = julgm4_b / (distm4_b - suspm4_b) × 100
```

O resultado é gravado em Resumo_Tribunais.csv, com uma linha por tribunal.

3.4 Opção 3 — Pesquisar por Município

Regras obrigatórias do input:

- O nome do município deve ser digitado inteiramente em MAIÚSCULAS.
-
- Espaços no nome são substituídos por underline no nome do arquivo gerado (ex: "SAO PAULO" → SAO_PAULO.csv).
- O município deve existir exatamente como está grafado no campo municipio_oj dos CSVs.

Exemplo de inputs válidos:

```
BRASILIA
CEILANDIA
GUARA
SAO PAULO
```

O programa percorre todos os 27 arquivos e grava em <MUNICIPIO>.csv apenas os registros cujo campo municipio_oj corresponda exatamente ao nome digitado.

4. Funções Auxiliares

Funções de suporte declaradas em lista.h e implementadas em lista.c, responsáveis por operações de infraestrutura do sistema.

4.1 LimparTerminal

`void LimparTerminal()`	
Categoria	Auxiliar — Interface
Descrição	Limpa o terminal de acordo com o sistema operacional. Desenvolvida no Linux Zorin OS: em tempo de compilação, a diretiva #ifdef _WIN32 seleciona o comando cls (Windows) ou clear (Linux/macOS). Garante uma interface limpa entre as operações do menu.
Retorno	void — sem retorno
Observações	Dependente do SO. Testada e otimizada para Linux Zorin. Funciona corretamente no Windows, mas o comportamento visual pode variar.

4.2 Pausar

`void Pausar()`	
Categoria	Auxiliar — Interface
Descrição	Exibe a mensagem 'Pressione ENTER para continuar...' e aguarda o usuário pressionar Enter antes de prosseguir. Utiliza dois getchar() para consumir o newline residual do scanf anterior e o Enter do usuário.
Retorno	void — sem retorno

4.3 InicializarLista

`void InicializarLista(Lista *L)`	
Categoria	Auxiliar — Gerência de Lista
Descrição	Inicializa a lista zerando o Tamanho e configurando a Capacidade como MAX_TRIBUNAIS. Deve ser chamada antes de qualquer uso da lista para garantir estado consistente.
Parâmetros	`Lista *L` — Ponteiro para a lista a ser inicializada
Retorno	void — sem retorno
Observações	Chamada no main.c antes de cada operação do menu.

4.4 ListaCheia

`int ListaCheia(const Lista *L)`	
Categoria	Auxiliar — Gerência de Lista
Descrição	Verifica se a lista atingiu sua capacidade máxima comparando Tamanho com Capacidade.
Parâmetros	`const Lista *L` — Ponteiro constante para a lista a ser verificada
Retorno	1 se a lista estiver cheia; 0 caso contrário.

4.5 ListaVazia

`int ListaVazia(const Lista *L)`	
Categoria	Auxiliar — Gerência de Lista
Descrição	Verifica se a lista não contém nenhum elemento (Tamanho == 0).
Parâmetros	`const Lista *L` — Ponteiro constante para a lista
Retorno	1 se a lista estiver vazia; 0 caso contrário.

4.6 AdicionarTribunal

`int AdicionarTribunal(Lista *L, Tribunal A)`	
Categoria	Auxiliar — Gerência de Lista
Descrição	Insere um registro Tribunal no final da lista e incrementa Tamanho. Verifica internamente se a lista está cheia antes de inserir.
Parâmetros	`Lista *L` — Ponteiro para a lista de destino `Tribunal A` — Cópia do registro a ser inserido
Retorno	1 em caso de sucesso; 0 se a lista estiver cheia.

4.7 CarregarCSV

`int CarregarCSV(Lista *L, const char *Nome_Arquivo, Tribunal t)`	
Categoria	Auxiliar — I/O
Descrição	Abre um arquivo CSV, pula o cabeçalho e lê cada linha usando sscanf com formato misto de campos entre aspas e campos numéricos. Adiciona cada registro lido à lista via AdicionarTribunal. Atenção: campos como mesano_cnm1 e mesano_sent podem conter o valor NA sem aspas nos CSVs — linhas com esse padrão podem não ser parseadas corretamente pelo sscanf dependendo da versão.
Parâmetros	`Lista *L` — Ponteiro para a lista de destino `const char *Nome_Arquivo` — Nome do arquivo CSV a ser lido `Tribunal t` — Variável local usada como buffer de leitura
Retorno	Número de registros lidos com sucesso; -1 em caso de falha ao abrir o arquivo.

4.8 EscreverCSV

`int EscreverCSV(Lista *L, Tribunal t, const char *Nome_Arquivo)`	
Categoria	Auxiliar — I/O
Descrição	Abre o arquivo de destino em modo append ('a') e grava todas as linhas da lista usando fprintf com o mesmo formato dos arquivos de origem. Cada campo string é envolvido em aspas duplas; campos numéricos são gravados sem aspas.
Parâmetros	`Lista *L` — Ponteiro para a lista com os dados a gravar `Tribunal t` — Parâmetro não utilizado diretamente (mantido por assinatura) `const char *Nome_Arquivo` — Nome do arquivo CSV de destino
Retorno	Número de registros gravados.
Observações	Utiliza modo 'a' (append). O cabeçalho é escrito separadamente por EscreverCabecalhoConcatenado antes desta função.

4.9 EscreverCabecalhoConcatenado

`int EscreverCabecalhoConcatenado(int comando, const char *NomeArquivo)`	
Categoria	Auxiliar — I/O
Descrição	Quando comando == 0 (primeira iteração), abre o arquivo em modo 'w' (criando ou zerando o arquivo), grava o cabeçalho completo com os 33 nomes de campo e fecha o arquivo. Nas iterações seguintes (comando != 0), não faz nada.
Parâmetros	`int comando` — 0 para escrever o cabeçalho; qualquer outro valor para pular `const char *NomeArquivo` — Nome do arquivo de destino
Retorno	Sempre retorna 1.
Observações	O arquivo é aberto em 'w' apenas para o cabeçalho e depois em 'a' pelo EscreverCSV, evitando sobrescrever dados.

4.10 EscreverCabecalhoResumido

`int EscreverCabecalhoResumido(int comando, const char *NomeArquivo)`	
Categoria	Auxiliar — I/O
Descrição	Idêntico ao EscreverCabecalhoConcatenado, mas grava um cabeçalho resumido com 7 campos: sigla_tribunal, julgados_2026, Meta1, Meta2A, Meta2Ant, Meta4A, Meta4B.
Parâmetros	`int comando` — 0 para escrever o cabeçalho; qualquer outro valor para pular `const char *NomeArquivo` — Nome do arquivo de destino
Retorno	Sempre retorna 1.

5. Funções Principais

Funções de alto nível que orquestram as operações do menu, combinando as funções auxiliares para realizar as tarefas completas.

5.1 ConcatenarDados

`int ConcatenarDados(Lista *L, Tribunal T, const char *NomeArquivo)`	
Categoria	Principal — Operação 1
Descrição	Itera sobre os 27 arquivos CSV dos TREs. Para cada arquivo: (1) escreve o cabeçalho apenas na primeira iteração, (2) carrega o arquivo na lista, (3) descarrega a lista no arquivo de destino e (4) reseta a lista para o próximo arquivo. Informa no terminal se houver déficit entre registros lidos e escritos.
Parâmetros	`Lista *L` — Ponteiro para a lista (reutilizada a cada arquivo) `Tribunal T` — Buffer de leitura passado ao CarregarCSV `const char *NomeArquivo` — Nome do arquivo de saída consolidado
Retorno	void — sem retorno (retorno declarado como int mas não implementado).
Observações	Ao final, imprime no terminal o nome do arquivo criado. A lista é resetada (Tamanho = 0) após cada arquivo para economizar memória.

5.2 GerarResumo

`int GerarResumo(Lista *L, Tribunal T, const char *NomeArquivo)`	
Categoria	Principal — Operação 2
Descrição	Para cada um dos 27 arquivos, carrega os dados, acumula os totais de todos os campos relevantes e calcula os cinco percentuais de meta usando as fórmulas do CNJ. Grava uma linha resumida por tribunal no arquivo de saída.
Parâmetros	`Lista *L` — Ponteiro para a lista `Tribunal T` — Buffer de leitura `const char *NomeArquivo` — Nome do arquivo de resumo de saída
Retorno	void — sem retorno (retorno declarado como int mas não implementado).
Observações	As somas são reiniciadas implicitamente a cada arquivo via reset de L->Tamanho. Atenção: as variáveis de soma não são zeradas entre arquivos no código atual — cada iteração acumula sobre o total anterior. Verifique se esse comportamento é o desejado para seu caso de uso.

5.3 PesquisarMunicipio

`void PesquisarMunicipio(Lista *L, Tribunal T, const char *NomeMunicipio)`	
Categoria	Principal — Operação 3
Descrição	Percorre todos os 27 arquivos e filtra apenas os registros cujo campo municipio_oj seja igual ao município informado. Os registros filtrados são gravados em um arquivo CSV nomeado como o próprio município (espaços substituídos por underline). O arquivo de destino é criado/zerado antes do loop e permanece aberto até o final.
Parâmetros	`Lista *L` — Ponteiro para a lista `Tribunal T` — Buffer de leitura `const char *NomeMunicipio` — Nome do município a pesquisar (em maiúsculas, sem acento)
Retorno	void — sem retorno.

Observações	A comparação é feita com strcmp, que é case-sensitive e sensível a acentos. O município deve ser digitado exatamente como aparece no campo municipio_oj dos CSVs (letras maiúsculas, sem acentos).

6. Fluxo de Execução

O diagrama abaixo resume o fluxo de chamadas para cada opção do menu:

Opção	Sequência de chamadas
[1]	`main → InicializarLista → ConcatenarDados → [EscreverCabecalhoConcatenado + CarregarCSV + EscreverCSV] × 27`
[2]	`main → InicializarLista → GerarResumo → [EscreverCabecalhoResumido + CarregarCSV + soma + fprintf] × 27`
[3]	`main → InicializarLista → PesquisarMunicipio → [EscreverCabecalhoConcatenado + CarregarCSV + strcmp + fprintf] × 27`

7. Integrantes

Nome	Registro Acadêmico
Anita Silva Moreira	UC2510177
Diogo Souza Moreira	UC25102288
Erick Teixeira dos Santos	UC25101750
Guilherme Augusto dos Santos	UC25100900
João Victor Veras Agapito	UC25102027

Orientador: **Prof. Marcelo Eustaquio** — Disciplina: Estrutura de Dados — Engenharia de Software — UCB, 2026.